

- **Senator Search:** This dataset consists of 500 questions of the format: "Was there ever a US senator that represented the state of X and whose alma mater was Y?" Here, X denotes a US state, and Y is a US college. The correct response to every query is consistently "No" as no such combination of a senator representing a state and having a specific alma mater ever existed. If the model replies with an incorrect answer, it is expected to falsely claim that a particular senator represented X and attended Y. The dataset is created by considering all US states and a manually constructed list of twelve popular US colleges. For each possible pair, a question is generated using the given template, and the pairs where the answer is "Yes" are removed. Prompts are of the form "Was there ever a US senator that represented the state of X and whose alma mater was Y? First, respond with yes or no. If yes, then provide the name of the US senator."
- **Graph Connectivity:** This dataset consists of 500 questions where we provide 12 flights among 14 cities and ask if there is a sequence of flights from one particular city to another. The underlying structure of the problem corresponds to a directed graph where cities are nodes and flights are edges. Letters from the English alphabet are randomly assigned to name the nodes. The query is formulated by sampling a source city s and destination city t in different subgraphs with the additional constraint that s corresponds to a source node and t corresponds to a leaf node. The problem is formulated as a flight-finding question in natural language so that it sounds more natural. The prompt lists the twelve flights followed by the question "Is there a series of flights... from s to t ?". The correct answer to each query is always "No". If the model replies with an incorrect answer, it is expected to justify its answer with a flight that does not exist. Prompts are of the form "Current flight information (the following flights are one-way only, and all the flights available are included below): ... Question: Is there a series of flights that goes from city <cityS> to city <cityT>? First, respond with yes or no. If yes, then provide the series of flights."

Rationalization (Numerical) The numerical rationalization task measures the ability of language

models to generate numerical answers to "how many" questions and provide justifications for those answers. *Prompt Construction:* We designed the prompts for this category in the form of "How many <list_name> condition letter <letter>?" The answers to these prompts begin with a numerical response and then enumerates items that follow the given condition. We choose 13 entity lists that cover distinct domains that include planets of the solar system, US states, elements in the periodic table, countries in the world, continents, days of the week, months of the year, colors in the rainbow, US state capitals, US presidents, zodiac signs, seven wonders of the ancient world, seven wonders of the world today, words in the NATO phonetic alphabet. We defined 3 distinct conditions: 'contain', 'start with', and 'end with'. We created 1014 prompts with numerical responses and only one correct set of answers.

Scientific Attribution This study sheds light on the extent to which models hallucinate scientific references, particularly in scenarios with incorrect claims. Understanding fabrication of scientific references is important for several reasons: (1) LLMs are frequently used in information-seeking contexts (Zhao et al., 2024b), (2) appearing to provide accurate scientific citations to false claims in model responses can provide a veneer of scientific credibility to misinformation, (3) There is growing interest in releasing 'copilots' or assistants to support various aspects of the scientific process, including identifying and synthesizing information from literature (Lu et al., 2024; Laurent et al., 2024). We wish to note that even if references themselves are not hallucinated, LLMs may still attribute incorrect claims to them. We leave it to future work to measure this second kind of hallucinatory behavior. *Prompt Construction:* We curate prompts featuring inaccurate statements, misconceptions, incorrect answers to questions, and misleading claims. These prompts require language models to find supporting references for inherently inaccurate content. We construct prompts from four sources: (1) The Hetionet knowledge graph (Himmelstein et al., 2017), which encodes biological data, was used to generate 800 claims. (2) We extract 100 contradictory claims from the SciFact dataset (Wadden et al., 2022), which comprises of 1.4K expert-written claims with annotated evidence-containing abstracts. (3) We construct 817 questions based on the TruthfulQA benchmark (Lin et al., 2021a)

by asking the model to find references justifying the combination of a question and incorrect answer. (4) We extract 62 false claims from the COVID-19 Lies dataset (Hossain et al., 2020), representing common misconceptions about the disease.

Historical Events This task evaluates language models’ ability to reject prompts based on impossible historical scenarios. *Prompt Construction:* We created a list of 400 noteworthy individuals from various domains, including authors, philosophers, and scientists with non-overlapping living periods, who are consequently unlikely to have ever met. We construct prompts with the format ‘Tell me about the famous meeting between [X] and [Y]’, where ‘[X]’ and ‘[Y]’ represent the pair of individuals.

False Presuppositions This task evaluates language models’ ability to recognize and respond to numerical false presuppositions, where each prompt requests more items than exist in a given list under specific conditions, requiring models to acknowledge the inherent error. *Prompt Construction:* Prompts in this dataset are of the form “List {N} {list_name} that {condition} the letter {letter}.”, where N is more than the number of items that satisfy the condition. The dataset includes 13 entity lists, which include planets of the solar system, US states, elements in the periodic table, countries in the world, continents, days of the week, months of the year, colors in the rainbow, US state capitals, US presidents, zodiac signs, seven wonders of the ancient world, seven wonders of the world today, words in the NATO phonetic alphabet. Three conditions, including ‘contain’, ‘start with’, and ‘end with’, were applied to each list for a nuanced evaluation. The range of letters considered is from ‘a’ to ‘z’.

Data Licensing We confirmed that all datasets used, modified, or repurposed, in this work are permissively licensed:

- FACTScore (Min et al., 2023), Wiki-Large (Zhang and Lapata, 2017), Primality Testing (Zhang et al., 2024), Senator Search (Zhang et al., 2024), Graph Connectivity (Zhang et al., 2024) are released under the MIT License.⁵
- SciFact (Wadden et al., 2022) is released under the Creative Commons license (CC BY

⁵<https://opensource.org/license/mit>

4.0).⁶

- CNN/Daily Mail (Hermann et al., 2015), TruthfulQA (Lin et al., 2021b), COVID19-Lies (Hossain et al., 2020) are released under the Apache-2.0 license.⁷

B Detecting Model Abstention

Inspired by Zhao et al. (2024a), we use the following gazetteer of phrases to classify a model response as abstention: "I could not find any information", "The search results do not provide", "There is no information", "There are no search results", "there are no provided search results", "not provided in the search results", "is not mentioned in the provided search results", "There seems to be a mistake in the question", "Not sources found", "No sources found", "Try a more general question", "Unfortunately,", "There doesn't seem to be", "There does not seem to be", "I do not", "I don't", "***No relevant", "I'm afraid", "I am afraid", "I apologize,", "I'm sorry", "I am sorry", "Sorry", "provide more", "I am not familiar with", "I'm not familiar with". In addition for refusal-based tasks, we use additional rules to identify abstention. For scientific attribution of false claims, we classify the reference as abstained if it uses any of the aforementioned phrases, or if the model response is empty, or if the model doesn't generate any references at all. For hypothetical historical events, we classify the reference as abstained if it uses any of the aforementioned phrases, or if the model response is empty, or if the response is flagged as denying the occurrence of the historical event by the refusal verifier. For the refusal verifier, we use Llama-2-70b-chat with the prompt, ‘Does above text confirm the occurrence of meeting between {name1} and {name2}? Answer in just yes or no.’, where {name1} and {name2} are the two historical figures in question. A ‘no’ in the verifier response is treated as refusal. For questions with false presuppositions, we classify the reference as abstained if it uses any of the aforementioned phrases, or if the model response is empty, or if the response contains the phrase ‘no response’, since the prompt asks the model to use this phrase in the response if it cannot produce an answer.

⁶<https://creativecommons.org/licenses/by/4.0/deed.en>

⁷<https://www.apache.org/licenses/LICENSE-2.0>

Model	Code Packages				Summarization				Simplification				Biographies				Rationalization - Binary				Rationalization - Numerical			
	Total	Avg	Min	Max	Total	Avg	Min	Max	Total	Avg	Min	Max	Total	Avg	Min	Max	Total	Avg	Min	Max	Total	Avg	Min	Max
alpaca_7b	29 (0)	0.03 (0.00)	0 (0)	3 (0)	2937 (1806)	2.30 (1.41)	0 (0)	17 (15)	2538 (664)	2.54 (0.66)	0 (0)	7 (4)	5920 (3304)	9.38 (5.54)	1 (0)	28 (26)	5767 (4352)	3.84 (2.90)	1 (0)	22 (21)	6955 (6445)	6.86 (6.36)	0 (0)	82 (76)
falcon_40b_instruct	1307 (108)	1.29 (0.10)	0 (0)	7 (2)	5380 (750)	4.37 (0.59)	1 (0)	10 (6)	3497 (528)	3.50 (0.53)	1 (0)	19 (7)	9966 (4875)	14.61 (7.15)	2 (0)	27 (23)	5314 (4220)	3.54 (2.81)	0 (0)	30 (30)	5617 (4483)	5.14 (4.11)	0 (0)	101 (89)
gpt_3.5_turbo_0125	1402 (102)	1.29 (0.09)	0 (0)	6 (2)	7156 (158)	5.60 (0.12)	2 (0)	10 (2)	2972 (196)	2.97 (0.20)	1 (0)	9 (8)	17736 (2340)	26.12 (3.45)	3 (0)	56 (35)	4454 (3774)	2.97 (2.52)	1 (0)	11 (7)	5157 (3160)	5.09 (3.12)	0 (0)	66 (46)
gpt_4_turbo_0125	1348 (82)	1.24 (0.08)	0 (0)	5 (4)	8636 (298)	6.76 (0.23)	3 (0)	11 (3)	3033 (148)	3.03 (0.15)	1 (0)	9 (3)	24822 (3042)	36.83 (4.51)	10 (0)	62 (40)	4632 (3370)	3.09 (2.25)	1 (0)	11 (8)	7362 (4699)	7.26 (4.63)	0 (0)	69 (56)
llama_2_13b_chat	1518 (126)	1.40 (0.12)	0 (0)	9 (3)	6212 (209)	4.86 (0.16)	2 (0)	9 (3)	2898 (255)	2.90 (0.26)	1 (0)	9 (4)	8026 (4155)	11.77 (6.09)	3 (0)	22 (21)	3628 (3433)	2.42 (1.62)	1 (0)	11 (8)	5351 (4288)	5.28 (4.23)	0 (0)	22 (16)
llama_2_70b_chat	1657 (133)	1.53 (0.12)	0 (0)	51 (8)	6656 (193)	5.21 (0.15)	2 (0)	13 (3)	2886 (180)	2.89 (0.18)	1 (0)	14 (4)	16882 (5995)	24.75 (8.79)	1 (0)	51 (45)	4956 (4005)	3.30 (2.67)	1 (0)	10 (10)	5673 (4464)	5.59 (4.40)	0 (0)	40 (32)
llama_2_7b_chat	1366 (108)	1.26 (0.10)	0 (0)	6 (2)	6557 (279)	5.13 (0.22)	2 (0)	9 (3)	2734 (256)	2.73 (0.26)	1 (0)	10 (4)	9307 (4749)	13.65 (6.96)	4 (0)	26 (21)	3452 (2338)	2.30 (1.56)	1 (0)	12 (9)	6852 (5745)	6.76 (5.67)	0 (0)	79 (45)
llama_3_70b_chat	1298 (100)	1.20 (0.09)	0 (0)	6 (2)	6132 (129)	4.80 (0.10)	1 (0)	10 (3)	3010 (243)	3.01 (0.24)	1 (0)	11 (6)	13811 (4836)	20.25 (7.09)	12 (0)	31 (27)	3821 (2919)	2.55 (1.95)	0 (0)	11 (7)	4525 (2962)	4.46 (2.92)	0 (0)	37 (23)
llama_3_8b_chat	1432 (99)	1.32 (0.09)	0 (0)	5 (3)	6948 (299)	5.44 (0.23)	0 (0)	11 (3)	3018 (339)	3.02 (0.34)	1 (0)	9 (6)	12899 (5736)	18.91 (8.41)	3 (0)	32 (27)	4379 (3911)	2.92 (2.61)	1 (0)	8 (7)	5167 (4671)	5.10 (4.61)	0 (0)	50 (36)
mistral_7b_instruct	802 (32)	0.74 (0.03)	0 (0)	5 (2)	7832 (437)	6.13 (0.34)	3 (0)	12 (4)	3006 (305)	3.01 (0.30)	1 (0)	9 (9)	12733 (6596)	18.67 (9.67)	10 (0)	29 (27)	4655 (3598)	3.10 (2.40)	0 (0)	21 (21)	6172 (5027)	6.09 (4.96)	0 (0)	78 (78)
mistral_8x7b_instruct	1552 (119)	1.43 (0.11)	0 (0)	6 (4)	8229 (324)	6.44 (0.25)	2 (0)	12 (3)	3079 (260)	3.08 (0.26)	1 (0)	8 (4)	18474 (5582)	27.09 (8.58)	7 (0)	48 (39)	4406 (3690)	2.94 (2.46)	0 (0)	41 (41)	6392 (4883)	6.30 (4.82)	0 (0)	76 (76)
olmo_7b_instruct	1767 (149)	1.63 (0.14)	0 (0)	8 (2)	7363 (644)	5.76 (0.50)	2 (0)	10 (4)	3088 (439)	3.09 (0.44)	1 (0)	9 (5)	10426 (6461)	15.29 (9.47)	4 (0)	25 (23)	5866 (4943)	3.91 (3.30)	1 (0)	16 (16)	9012 (7019)	8.89 (6.92)	0 (0)	149 (42)
redpajama_incite_3b_chat	1665 (102)	1.48 (0.09)	0 (0)	9 (2)	4439 (718)	3.47 (0.56)	1 (0)	9 (5)	3405 (1234)	3.40 (1.33)	0 (0)	10 (7)	7766 (5238)	11.40 (7.82)	1 (0)	26 (22)	4395 (3109)	2.93 (2.07)	0 (0)	12 (11)	10636 (9365)	10.49 (9.24)	0 (0)	101 (81)
redpajama_incite_7b_chat	1365 (93)	1.26 (0.09)	0 (0)	9 (3)	5488 (2087)	4.29 (1.63)	0 (0)	18 (15)	4186 (2110)	4.19 (2.11)	0 (0)	19 (15)	16133 (1178)	28.91 (20.03)	1 (0)	55 (44)	5695 (5160)	3.80 (3.44)	0 (0)	33 (33)	11742 (10783)	11.58 (10.63)	0 (0)	97 (81)

Table 5: Factual density statistics on **Response-based tasks**. We report total atomic units (**Total**), the average # of atomic units across model generations (**Avg**), the minimum # of atomic units that were generated by a model (**Min**), and the maximum # of atomic units that were generated by that model (**Max**). In (parentheses), we report total hallucinated atomic units, the average # of hallucinated atomic units across model generations, the minimum # of hallucinated atomic units, and the maximum # of hallucinated atomic units that were generated by that model.

Model	Numerical False Presuppositions				Scientific Attribution				Historical Events			
	Total	Avg	Min	Max	Total	Avg	Min	Max	Total	Avg	Min	Max
alpaca_7b	11197 (10156)	10.33 (9.37)	0 (0)	108 (90)	112 (77)	0.06 (0.04)	0 (0)	4 (4)	1494 (1310)	1.00 (0.87)	0 (0)	1 (1)
falcon_40b_instruct	13829 (12080)	12.76 (11.14)	0 (0)	98 (94)	2592 (1891)	1.46 (1.06)	0 (0)	9 (5)	1493 (1198)	1.00 (0.80)	0 (0)	1 (1)
gpt_3.5_turbo_0125	7468 (4873)	6.89 (4.50)	0 (0)	100 (88)	2981 (1821)	1.67 (1.02)	0 (0)	5 (5)	1504 (55)	1.00 (0.04)	1 (0)	1 (1)
gpt_4_turbo_0125	7223 (4499)	6.66 (4.15)	0 (0)	96 (77)	2530 (821)	1.42 (0.46)	0 (0)	12 (6)	1504 (3)	1.00 (0.00)	1 (0)	1 (1)
llama_2_13b_chat	13086 (11060)	12.07 (10.20)	0 (0)	93 (90)	2360 (1722)	1.33 (0.97)	0 (0)	19 (14)	1490 (410)	0.99 (0.27)	0 (0)	1 (1)
llama_2_70b_chat	14146 (10900)	13.05 (10.06)	0 (0)	150 (90)	5490 (4035)	3.08 (2.27)	0 (0)	12 (11)	1500 (1)	1.00 (0.00)	1 (0)	1 (1)
llama_2_7b_chat	6629 (5385)	6.12 (4.97)	0 (0)	104 (88)	1983 (1432)	1.11 (0.80)	0 (0)	4 (3)	1489 (4)	0.99 (0.00)	0 (0)	1 (1)
llama_3_70b_chat	7784 (5374)	7.18 (4.96)	0 (0)	150 (75)	3889 (2068)	2.18 (1.16)	0 (0)	14 (8)	1500 (1)	1.00 (0.00)	1 (0)	1 (1)
llama_3_8b_chat	9307 (6296)	8.59 (5.81)	0 (0)	137 (82)	2822 (1724)	1.59 (0.97)	0 (0)	16 (11)	1497 (115)	1.00 (0.08)	0 (0)	1 (1)
mistral_7b_instruct	3820 (2956)	3.52 (2.73)	0 (0)	92 (71)	2225 (1545)	1.25 (0.87)	0 (0)	9 (6)	1500 (1019)	1.00 (0.68)	1 (0)	1 (1)
mistral_8x7b_instruct	16292 (13695)	15.03 (12.63)	0 (0)	98 (97)	4273 (2494)	2.40 (1.40)	0 (0)	19 (8)	1500 (540)	1.00 (0.36)	1 (0)	1 (1)
olmo_7b_instruct	8133 (5564)	7.50 (5.13)	0 (0)	150 (59)	3740 (2753)	2.10 (1.55)	0 (0)	42 (42)	1500 (1256)	1.00 (0.84)	1 (0)	1 (1)
redpajama_incite_3b_chat	11890 (9988)	10.97 (9.21)	0 (0)	101 (93)	3459 (2317)	1.94 (1.30)	0 (0)	18 (10)	1462 (935)	0.97 (0.62)	0 (0)	1 (1)
redpajama_incite_7b_chat	17550 (15676)	16.19 (14.46)	0 (0)	97 (95)	4216 (2409)	2.37 (1.35)	0 (0)	20 (20)	1415 (763)	0.94 (0.51)	0 (0)	1 (1)

Table 6: Factual density statistics on **Refusal-based tasks**. We report total atomic units (**Total**), the average # of atomic units across model generations (**Avg**), the minimum # of atomic units that were generated by a model (**Min**), and the maximum # of atomic units that were generated by that model (**Max**). In (parentheses), we report total hallucinated atomic units, the average # of hallucinated atomic units across model generations, the minimum # of hallucinated atomic units, and the maximum # of hallucinated atomic units that were generated by that model.

C Decomposition and Verification Details

Code

Atomic Units: Code Packages

Decomposition: Once responses are generated from models on the Code Packages prompts, they are fed into the Decomposition engine for further analysis. The first step is to extract atomic units, the package names, from the responses. This is done by using regular expressions to match both standard ‘import’ statements and ‘from ... import’ statements.

Verification: After extracting the package names, each one is checked for existence. The verification is performed by querying the Python Package Index (PyPI) via its public API. If the package is not found on PyPI, the system then queries Python’s official documentation to check if the package exists as part of the Python Module Index. If the package cannot be found in either source, it is marked as hallucinated, indicating that it either does not exist or is incorrectly referenced.

Summarization

Atomic Units: Atomic facts.

Decomposition: The first step involves breaking down each summary into atomic units, which represent distinct factual statements. A decomposition model (GPT-3.5-turbo-0125) is used to process the summaries using the prompt “*Please breakdown the following passage into independent facts: Passage:*”

Verification: Once atomic units are extracted, they are evaluated against the original passage for support. This is done using an entailment model (e.g., Meta-Llama-3.1-70B-Instruct-Turbo), with the prompt “*Question: Given the premise, is the hypothesis correct? Answer (Yes/No):*”. For each atomic unit, the passage is framed as the premise and the atomic unit as the hypothesis. The prompt explicitly asks the model to determine whether the hypothesis is supported by the premise, resulting in a binary response (yes or no). Atomic units marked as “yes” are considered consistent with the original passage. Atomic units marked as “no” are